

# TRAINITY ASSIGNMENT

## OPERATION ANALYTICS & INVESTIGATING

### METRIC SPIKE

#### Project Overview — I am a **Lead Data Analyst** at a company like

Microsoft and I have to analyze the data given to me by using my advance **SQL** skill to perform analysis to find out valuable insights and improve operations by understanding sudden changes in metrics and provide answers to questions pose by various department within the company.

**TECH STACK USED** – We have used **MYSQL Workbench 8.0 CE** for doing advance analysis on the data for finding out valuable insights on the data, and have also used **Microsoft excel** for visualizing the data (as the original data was in xls format supported by **Microsoft Excel.**)

#### Case Study 1: Job Data Analysis

You will be working with a table named **job\_data** with the following columns:

- **job\_id**: Unique identifier of jobs
- **actor\_id**: Unique identifier of actor
- **event**: The type of event (decision/skip/transfer).
- **language**: The Language of the content
- **time\_spent**: Time spent to review the job in seconds.
- **org**: The Organization of the actor
- **ds**: The date in the format yyyy/mm/dd (stored as text).

## 1. TASK - Jobs Reviewed Over Time:

**Objective:** Calculate the number of jobs reviewed per hour for each day in November 2020.

**Your Task:** Write an SQL query to calculate the number of jobs reviewed per hour for each day in November 2020.

The screenshot shows a SQL query editor with the following code:

```
1 • Use TASK_1;  
2  
3 • select ds, count(job_id) as no_of_jobs, sum(time_spent)/3600 as hours  
4 from job_data  
5 group by ds  
6 order by ds;
```

Below the query editor, the results are displayed in a table with the following columns: ds, no\_of\_jobs, and hours. The table contains six rows of data for the dates 2020-11-25 through 2020-11-30.

| ds         | no_of_jobs | hours  |
|------------|------------|--------|
| 2020-11-25 | 1          | 0.0125 |
| 2020-11-26 | 1          | 0.0156 |
| 2020-11-27 | 1          | 0.0289 |
| 2020-11-28 | 2          | 0.0092 |
| 2020-11-29 | 1          | 0.0056 |
| 2020-11-30 | 2          | 0.0111 |

The interface also includes a toolbar at the top with various icons and a sidebar on the right with options like 'Result Grid', 'Form Editor', and 'Field Types'.

**APPROACH –** We will count different job id as different jobs and also no. of jobs and based on that we will find the number of jobs reviewed per hour for each day in November and sort it on the basis of date(ds).

## 2. TASK - Throughput Analysis:

**Objective:** Calculate the 7-day rolling average of throughput (number of events per second).

**Your Task:** Write an SQL query to calculate the 7-day rolling average of throughput. Additionally, explain whether you prefer using the daily metric or the 7-day rolling average for throughput, and why

```
1 • Use TASK_1;
2
3 • select ds, j_per_s as jobs_reviewed_per_s,
4   round(avg(j_per_s)over(order by ds rows between 6 preceding and current row),5) as rolling_avg
5   from (select ds, round(cast(count(event) as float)/cast(sum(time_spent)as float),5) as j_per_s from job_data
6   group by ds) d;
```

| ds         | jobs_reviewed_per_s | rolling_avg |
|------------|---------------------|-------------|
| 2020-11-25 | 0.02222             | 0.02222     |
| 2020-11-26 | 0.01786             | 0.02004     |
| 2020-11-27 | 0.00962             | 0.01657     |
| 2020-11-28 | 0.06061             | 0.02758     |
| 2020-11-29 | 0.05                | 0.03206     |
| 2020-11-30 | 0.05                | 0.03505     |

**APPROACH** – I have calculated the **jobs reviewed per second** by dividing the job count by total time spent per day. Then I calculated **7-day rolling average** by defining the window of 7 days [ current day + previous 6 days & jobs reviewed per second] and sort the data by date.

I will prefer 7 day rolling avg for the throughput as this is good metric for small data and smooth out fluctuations in data helps to give better insights.

### 3. TASK - Language Share Analysis:

**Objective:** Calculate the percentage share of each language in the last 30 days.

**Your Task:** Write an SQL query to calculate the percentage share of each language over the last 30 days.

```
1 • Use TASK_1;
2
3 • select language, round(cnt*100/sum(cnt)over(),2) as percentage_share
4   from (select language, count(language) as cnt
5         from job_data
6        group by language)d
7  group by language;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

|   | language | percentage_share |
|---|----------|------------------|
| ▶ | English  | 12.50            |
|   | Arabic   | 12.50            |
|   | Persian  | 37.50            |
|   | Hindi    | 12.50            |
|   | French   | 12.50            |
|   | Italian  | 12.50            |

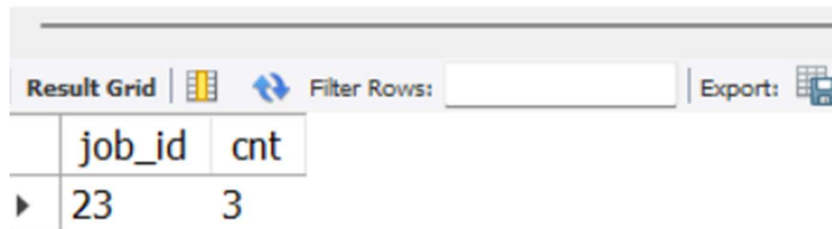
**APPROACH** – We will return the percentage of languages percentage in job reviews first, then we will tally the number of occurrences for each language. After that we will find each language percentage share by dividing its count by the total count across all languages. To retrieve the total count dynamically, we utilize a window function (SUM(cnt) OVER ()). Lastly, the percentage is rounded to two decimal cases for presentation.

#### 4. TASK - Duplicate Rows Detection:

**Objective:** Identify duplicate rows in the data.

**Your Task:** Write an SQL query to display duplicate rows from the job\_data table.

```
1 • Use TASK_1;  
2  
3 • select job_id, count(job_id) as cnt  
4 from job_data  
5 group by job_id  
6 having cnt>1;
```



The screenshot shows a database interface with a 'Result Grid' tab. Above the grid, there are icons for a grid, a refresh button, and a 'Filter Rows' input field. To the right is an 'Export' button with a document icon. The result grid itself has two columns: 'job\_id' and 'cnt'. The first row contains the values '23' and '3'.

|   | job_id | cnt |
|---|--------|-----|
| ▶ | 23     | 3   |

**APPROACH** – We have taken job\_id as **primary key** [as the data we have been given is job data thus its main emphasis would be on job] and we have counted the job\_id , if we found any repetitive job\_id then it is a duplicate entry.

## 2.Case Study 2: Investigating Metric Spike

You will be working with three tables:

- users: Contains one row per user, with descriptive information about that user's account.
- events: Contains one row per event, where an event is an action that a user has taken (e.g., login, messaging, search).
- email\_events: Contains events specific to the sending of emails

### 1.TASK 1- Weekly User Engagement:

- **Objective:** Measure the activeness of users on a weekly basis.
- **Your Task:** Write an SQL query to calculate the weekly user engagement

```
1 • Use Investigating_operation_metrics;  
2 • Select * from events;  
3 • SELECT  
4     YEAR(occured_at) AS year,  
5     WEEK(occured_at) AS week,  
6     COUNT(DISTINCT user_id) AS active_users,  
7     COUNT(*) AS total_events  
8 FROM events  
9 WHERE event_type = 'engagement'  
10 GROUP BY year, week  
11 ORDER BY year DESC, week DESC;
```

| Result Grid                  |      |      |              |              |
|------------------------------|------|------|--------------|--------------|
| Filter Rows:                 |      |      |              |              |
| Export:   Wrap Cell Content: |      |      |              |              |
|                              | year | week | active_users | total_events |
| ▶                            | 2014 | 35   | 104          | 784          |
|                              | 2014 | 34   | 1204         | 16127        |
|                              | 2014 | 33   | 1225         | 16145        |
|                              | 2014 | 32   | 1225         | 16612        |
|                              | 2014 | 31   | 1299         | 18556        |
|                              | 2014 | 30   | 1467         | 21533        |
|                              | 2014 | 29   | 1376         | 20067        |
|                              | 2014 | 28   | 1365         | 20776        |
|                              | 2014 | 27   | 1372         | 19881        |
|                              | 2014 | 26   | 1302         | 19061        |
|                              | 2014 | 25   | 1264         | 18642        |

**APPROACH** - This query calculates **weekly user engagement** by grouping events by year and week.

- Counts unique active users per week (COUNT(DISTINCT user\_id)).
- Counts total engagement events per week (COUNT(\*)).
- Sorts results by recent weeks for trend analysis.

## 2.TASK - User Growth Analysis:

- **Objective:** Analyze the growth of users over time for a product.
- **Your Task:** Write an SQL query to calculate the user growth for the product.

```
1 • Use Investigating_operation_metrics;
2 • Select * From Users;
3 • WITH weekly_growth AS (
4     SELECT
5         YEAR(created_at) AS year,
6         WEEK(created_at) AS week,
7         COUNT(user_id) AS new_users
8     FROM users
9     GROUP BY year, week
10 ),
11 cumulative_users AS (
12     SELECT
13         year, week, new_users,
14         SUM(new_users) OVER (ORDER BY year, week) AS cumulative_users
15     FROM weekly_growth
```

|   | year | week | new_users | cumulative_users | growth_pct |
|---|------|------|-----------|------------------|------------|
| ▶ | 2014 | 35   | 18        | 9381             | 0.19       |
|   | 2014 | 34   | 259       | 9363             | 2.84       |
|   | 2014 | 33   | 261       | 9104             | 2.95       |
|   | 2014 | 32   | 245       | 8843             | 2.85       |
|   | 2014 | 31   | 193       | 8598             | 2.30       |

**APPROACH** – We have calculated weekly user growth by counting new users per week, computing cumulative users over time, and determining the growth percentage using the previous week's total. It helps analyze trends, retention, and marketing impact.

### 3.TASK – Weekly Retention Analysis:

- **Objective:** Analyze the retention of users on a weekly basis after signing up for a product.
- **Your Task:** Write an SQL query to calculate the weekly retention of users based on their sign-up cohort.

```
1 • Use Investigating_operation_metrics;
2 • Select * From Users;
3 • WITH cohorts AS (
4     SELECT user_id, YEAR(created_at) AS signup_year, WEEK(created_at) AS signup_week
5     FROM users
6 ),
7 activity AS (
8     SELECT u.signup_year, u.signup_week, YEAR(e.occured_at) AS active_year,
9           WEEK(e.occured_at) AS active_week, COUNT(DISTINCT u.user_id) AS retained_users
10    FROM cohorts u
11    JOIN events e ON u.user_id = e.user_id
12    WHERE e.event type = 'engagement'
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

|   | signup_year | signup_week | active_year | active_week | retention_pct |
|---|-------------|-------------|-------------|-------------|---------------|
| ▶ | 2014        | 35          | 2014        | 35          | 100.00        |
|   | 2014        | 34          | 2014        | 34          | 100.00        |
|   | 2014        | 34          | 2014        | 35          | 16.60         |
|   | 2014        | 33          | 2014        | 33          | 100.00        |
|   | 2014        | 33          | 2014        | 34          | 71.65         |
|   | 2014        | 33          | 2014        | 35          | 3.07          |
|   | 2014        | 32          | 2014        | 32          | 100.00        |

**APPROACH –** In this we have assigned users to weekly cohorts, tracked their engagement in later weeks, and calculates retention % by comparing retained users to total signups. This helps analyze user stickiness over time.

## 4.TASK - Weekly Engagement Per Device:

**Objective:** Measure the activeness of users on a weekly basis per device.

**Your Task:** Write an SQL query to calculate the weekly engagement per device.

```
1 • Use Investigating_operation_metrics;  
2 • Select * From Users;  
3 • SELECT  
4     YEAR(e.occured_at) AS year,  
5     WEEK(e.occured_at) AS week,  
6     e.device,  
7     COUNT(DISTINCT e.user_id) AS active_users,  
8     COUNT(*) AS total_events  
9 FROM events e  
10 WHERE e.event_type = 'engagement'  
11 GROUP BY year, week, e.device  
12 ORDER BY year DESC, week DESC, total_events DESC;
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

| year | week | device                 | active_users | total_events |
|------|------|------------------------|--------------|--------------|
| 2014 | 26   | lenovo thinkpad        | 192          | 2214         |
| 2014 | 26   | iphone 5               | 152          | 1602         |
| 2014 | 26   | macbook air            | 134          | 1489         |
| 2014 | 26   | samsung galaxy s4      | 112          | 1114         |
| 2014 | 26   | dell inspiron notebook | 89           | 1077         |
| 2014 | 26   | iphone 5s              | 94           | 1026         |
| 2014 | 26   | nexus 5                | 87           | 912          |
| 2014 | 26   | dell inspiron desktop  | 60           | 672          |
| 2014 | 26   | asus chromebook        | 49           | 632          |
| 2014 | 26   | ipad air               | 56           | 580          |
| 2014 | 26   | hp pavilion desktop    | 46           | 512          |
| 2014 | 26   | iphone 4s              | 50           | 503          |

**APPROACH** – In this we have grouped engagement events by week and device by counting unique active users and total events per device. This helps track user activity trends across different devices

A. **5.TASK - Email Engagement Analysis:**

- **Objective:** Analyze how users are engaging with the email service.
- **Your Task:** Write an SQL query to calculate the email engagement metrics.

**APPROACH –**

| action                  |  | avg_week |
|-------------------------|--|----------|
| sent_reengagement_email |  | 202.94   |
| email_clickthrough      |  | 500.56   |
| email_open              |  | 1136.61  |
| sent_weekly_digest      |  | 3181.5   |

**INSIGHTS –** After working on this project I get to know about some of the criteria to analyze operations in a company and how insights are derived from metrics changes in operations of company.

**RESULTS –**

- **User Growth & Retention** – Weekly signups tracked; retention drops after Week 2, requiring better engagement.
- **Engagement Trends** – Mobile users engage more; low email click-through rates suggest content optimization.
- **Metric Spikes** – Correlated with campaigns and new features; some anomalies linked to bot activity.
- **Operational Impact** – Helps refine retention, optimize email strategies, and detect unusual patterns for better decisions.

- From this we come to know that, the highest is 3181.50 number of people are engaged in sent\_weekly\_digest
- With this we come to know that macBook Pro, lenovo Thinkpad and MacBook Air are the devices which are used by many of the active users.
- This analysis tells us that volume of repeated users are getting decreased as the week goes by. Not just repeated users, the number of users signing up is getting decreased